PROJECT REPORT

VoiceOps 2026 - ZentroMart

Customer-support voice and conversational AI agent for a synthetic retail dataset

Project focus	Policy-aware retrieval, verified order support, ticket escalation, and voice integration
Source	Project README and implementation status documented in the repository
Report date	30 October 2026

Prepared as a professional summary of the documented implementation, validation approach, operating instructions, and remaining work.

1. Executive Summary

VoiceOps 2026 - ZentroMart is a phased customer-support agent built around a synthetic ZentroMart dataset. The project combines a read-only dataset repository, prompt-only and retrieval-augmented chat evaluation, policy-aware evidence selection, conversational state management, identity-gated order support, ticket escalation, and an offline voice integration foundation.

The documented implementation is deliberately evaluation-oriented. It separates model output from expected answers and evaluation labels, preserves evidence provenance, records structured failures, and uses deterministic offline tests for conversation and voice behavior. The README identifies the data foundation, prompt-only chatbot, RAG layer, conversational agent, and offline voice foundation as complete, while live audio validation and later comparative evaluation remain pending.

Overall assessment: the project has a strong, testable foundation for a customer-support prototype. Its most important next step is controlled live validation of the configured providers, followed by end-to-end quality measurement across baseline, RAG, multi-turn, and voice scenarios.

Area	Documented status	Key evidence
Data foundation	Complete	Read-only repository; 9 CSVs, 21 KB documents; structural and relational validation.
Prompt-only chatbot	Complete	52-question batch runner with structured scoring and explicit API failure records.
Policy-aware RAG	Complete	Section indexing, metadata retention, policy precedence, superseded-content exclusion.
Conversational support	Offline implementation complete	Isolated conversation state, identity verification, order lookup, ticket deduplication, handover.
Voice foundation	Offline complete; live validation pending	Gnani STT/TTS adapters, retries, streaming lifecycle, single agent turn per STT completion.
Future evaluation	Planned	Prompt-only vs RAG and multi-turn evaluation; live voice verification.

2. Objectives and Scope

The project is designed to provide a reliable customer-support workflow over a controlled synthetic retail environment. The implementation scope documented in the README includes:

- Preserving the supplied dataset without modification while loading identifiers and references as text.
- Parsing knowledge-base frontmatter and section identifiers for traceable retrieval.
- Answering practice questions through a configured Gemini or Groq provider without exposing expected answers to the model.
- Adding retrieval that prioritizes current policy content and excludes superseded sections from customer answers.
- Maintaining isolated multi-turn context by conversation ID, with verification before disclosing order details.
- Creating tickets idempotently and returning structured handover, evidence, and failure information.
- Providing an official Gnani SDK-based STT/TTS integration foundation, including streaming adapters and bounded retry behavior.

Out of scope or not yet demonstrated

- No live Gemini conversation evaluation is performed by the current tests.

- The current voice implementation does not perform a live provider request.
- Streaming retries cannot claim exactly-once provider-side work because the SDK does not guarantee it.
- The documented 52/52 result is evidence coverage for active requirements, not 100% answer accuracy.

3. Solution Architecture

The solution is organized as a layered support system. The data and knowledge layers provide controlled evidence; the chatbot and RAG layers generate answers; the conversational layer manages state and tools; and the voice layer translates between audio and agent turns.

Layer	Responsibility	Controls and outputs
Dataset repository	Discovers and loads supplied CSV and KB content without editing it.	Text-preserving identifiers, order preservation, parsed frontmatter and section IDs.
Chat / LLM client	Generates responses through Gemini by default or Groq when configured.	Explicit API-key failure, bounded retries, redacted error records, no evaluation-label leakage.
RAG retrieval	Indexes KB sections and selects evidence using lexical retrieval and policy precedence.	Current policy first; superseded content excluded; evidence IDs and metadata retained.
Conversation agent	Processes turns, verifies identity, invokes tools, and stores context per conversation.	One retry for verification, ticket idempotency, structured failure and handover records.
Voice service	Transcribes audio, executes one conversational turn, and synthesizes response audio.	Official Gnani SDK contracts, stage-specific retry rules, streaming completion events.

Key operational invariants

- Order details are disclosed only after successful phone-last-four or delivery-pincode verification.
- The conversational agent is not retried by the voice layer, preventing duplicate order lookups or ticket creation.
- A TTS retry is permitted only before the first audio chunk is delivered; failures after partial delivery are propagated.
- Evidence recall is evaluated only when an active section is required and the turn actually invokes the RAG flow.

4. Implementation Progress

Phase	Capability	Status	Documented result
1	Data foundation	Complete	Validation expects 9 CSVs, 21 KB documents, and verification checks to pass.
2	Prompt-only text chatbot	Complete	52-question batch output includes answers, decisions, required-fact checks, forbidden-claim checks, and summary.
3	Policy-aware RAG	Complete	Offline retrieval validation covers 52/52 active evidence requirements on the practice-question dataset.
4	Conversational support and tools	Offline implementation complete	Tests cover isolation, follow-ups, identity, refusal, escalation, ticket failures, handover, APIs, and provenance.
5	Gnani voice integration foundation	Offline complete; live pending	STT/TTS and WebSocket streaming adapters are implemented and tested with mocks.
6	Comparative and multi-turn evaluation	Planned	Prompt-only vs RAG and broader multi-turn evaluation remain to be completed.

5. Evaluation and Validation

The project emphasizes reproducibility and metric separation. Runners produce JSON outputs containing per-question or per-turn results, model decisions, required-fact checks, forbidden-claim checks, evidence identifiers, tool outcomes, and summary metrics.

Validation area	Method	Interpretation
Dataset integrity	Structural and relational validator plus unit tests.	Confirms the supplied synthetic dataset is discoverable, readable, and internally consistent.
Prompt baseline	52-question batch through configured LLM provider.	Establishes baseline behavior; results depend on live provider availability and configuration.
RAG evidence	Offline retrieval check against gold evidence requirements.	52/52 active evidence requirements covered; this is evidence coverage, not answer accuracy.
Conversation behavior	Deterministic mocked RAG client and synthetic tool adapters.	Validates state, policy, identity, ticket, handover, evidence, context, and failure handling.
Voice behavior	Protocol-compatible mocks with no network requests.	Validates retry boundaries and stream lifecycle without claiming live provider performance.

Recommended interpretation of results

A clean evaluation run requires a configured model, non-zero scored results, and zero recorded errors for the relevant run. API failures are retained as unscored structured records rather than silently converted into incorrect answers. This distinction is important for reliable reporting and later comparison.

6. Security, Privacy, and Reliability Controls

Control	Implementation documented in README
Credential handling	API keys are read from environment variables, are not stored in source code, and are not written to output logs.
Error redaction	Provider failures record credential-redacted messages plus category, type, status, and API code.
Dataset safety	The supplied synthetic dataset is read-only and must not be edited.
Identity gating	Order details require successful phone-last-four or delivery-pincode verification.
Retry safety	Retries are bounded and limited to transient STT/TTS/provider failures; agent/tool stages are not retried by voice.
Ticket safety	Ticket creation is idempotent per conversation to reduce duplicate escalations.
Evidence governance	Current policy sections take precedence; superseded content is excluded from customer answers.

7. Operating Instructions

The repository README provides the following repeatable workflows. Commands should be run from the project directory with PYTHONPATH set to src.

Workflow	Purpose	Primary command or action
Install and configure	Install dependencies and configure provider credentials.	python -m pip install -r requirements.txt; copy .env.example to .env; set GEMINI_API_KEY or Groq settings.
Validate Phase 1	Check dataset structure and repository behavior.	python scripts\validate_phase1.py; python -m unittest discover -s tests
Run Step 1	Evaluate prompt-only chatbot over 52 questions.	python scripts\run_step1.py --output outputs\step1_results.json
Run Step 2	Compare baseline and policy-aware RAG.	python scripts\run_step2.py --output outputs\step2_results.json
Run Step 3	Score deterministic conversational support behavior.	python scripts\run_step3_evaluation.py
Run diagnostics	Test a single request or a three-question evaluation before a full run.	Use the small Python snippets documented in the README.

8. Risks and Recommended Next Steps

Priority	Risk / gap	Recommended next step
High	Live provider behavior is not yet demonstrated for conversation and voice.	Run controlled Gemini and Gnani tests with non-production credentials, capture latency/error distributions, and verify audio formats.
High	Evidence coverage does not measure answer correctness.	Complete baseline-vs-RAG scoring with answer quality, policy adherence, and unsupported-claim metrics.
Medium	Provider quotas, rate limits, and model availability can affect repeatability.	Record model, configuration, timestamps, retry outcomes, and environment details with each evaluation artifact.
Medium	Streaming failure before first chunk can repeat provider-side work.	Add operational telemetry and a request correlation ID; document duplicate-risk handling for production.
Medium	Multi-turn evaluation remains planned.	Expand deterministic scenarios and then compare against live, human-reviewed conversations.

9. Conclusion

VoiceOps 2026 - ZentroMart presents a thoughtful foundation for an auditable customer-support agent. The strongest aspects are explicit data preservation, traceable retrieval evidence, policy-aware ranking, identity-gated order disclosure, idempotent ticket escalation, and deliberate separation of live-provider failures from scored model behavior.

The project is ready for the next validation stage: controlled live testing and comparative evaluation. Before production-oriented use, the team should establish measured answer quality, latency, provider reliability, audio compatibility, observability, and operational procedures for partial streaming failures.

Source note: This report was prepared from the project README supplied in the repository and reflects the implementation status and limitations documented there as of 30 October 2026.